

T1 Acceso a ficheros

Tipos de ficheros

Desde el punto de vista de un programador solamente distinguiremos entre dos tipos de ficheros:

- **Ficheros de texto** cuando el contenido del fichero contenga exclusivamente caracteres de texto (podemos leerlo con un simple editor de texto)

Extensión Tipo de fichero

.txt Fichero de texto plano

.xml Fichero XML

.json Fichero de intercambio de información

.props Fichero de propiedades

.conf Fichero de configuración

.sql Script SQL

.srt Fichero de subtítulo

- **Ficheros binarios** cuando no estén compuestos exclusivamente de texto. Pueden contener imágenes, videos, ficheros, . . . aunque también podemos considerar un fichero binario a un fichero de Microsoft Word en el que sólo hayamos escrito algún texto puesto que, al almacenarse el fichero, el procesador de texto incluye alguna información binaria

Extensión Tipo de fichero

.pdf Fichero PDF

.jpg Fichero de imagen

.doc, .docx Fichero de Microsoft Word

.avi Fichero de video

.ppt, .pptx Fichero de PowerPoint

A veces, en ficheros binarios, podremos encontrarnos con las extensiones *.bin* o *.dat* para hacer referencia a ficheros que contienen información binaria en un formato que no está ampliamente difundido. Serán simplemente ficheros que una aplicación determinada es capaz de leer/escribir de una forma específica solo definida para dicha aplicación.

Propiedades del Sistema

En cualquier caso, para el acceso a ficheros independientemente del tipo de los mismos, conviene conocer el funcionamiento de las [System Properties](#) de las que dispone Java en su API. Éstas permiten acceder a propiedades de la configuración del sistema y, entre otras, podemos encontrarnos con algunas muy interesantes relacionadas con este tema:

- `"file.separator"` Obtiene el caracter, según el S.Operativo, para la separación de las rutas (/ ó \). También se puede utilizar la constante `File.separator`
- `"user.home"` Obtiene la ruta de la carpeta personal del usuario (que dependerá del S.Operativo en cada caso)
- `"user.dir"` Obtiene la ruta en la que se encuentra actualmente el usuario
- `"line.separator"` Obtiene el caracter que separa las líneas de un fichero de texto (difiere entre Windows/Linux)

En el caso de que se quiera acceder al valor de alguna de estas propiedades debe hacerse utilizando la llamada al método `System.getProperty(String)`

Tipos de Ficheros::

- Ficheros de texto:
 - Texto plano.
 - Ficheros de configuración, con parejas de variable=valor
 - Ficheros XML
 - Otros ficheros de texto plano
- Ficheros binarios (se guardan bytes, no imprimibles:
 - imágenes, sonido, vídeos, ficheros comprimidos ,...

Ficheros de texto

En esta parte vamos a trabajar con 3 tipos de ficheros de texto:

- **Ficheros de texto plano** que contendrán texto *libre* y donde podremos escribir sin respetar ningún tipo de formato
- **Ficheros de configuración** que contendrán información de configuración para una aplicación. Tienen un formato específico y Java además proporciona una API para trabajar más cómodamente con ellos
- **Ficheros XML** que contienen información y acompañada de etiquetas que le dan significado. Tienen unas reglas y formato más o menos definido y Java proporciona una API para trabajar con ellos

Existen más tipos de ficheros de texto también muy extendidos como `.json` y `.csv` pero no serán estudiados en esta parte.

Ficheros de texto plano

Los ficheros de texto son aquellos que únicamente contienen texto, por lo que pueden ser editados directamente con cualquier editor de texto plano (Bloc de Notas, notepad++, . . .). Se podría decir que son aquellos ficheros que podrían ser *leídos* por cualquier persona. Son aquellos que normalmente se almacenan con la extensión *.txt* pero también podríamos incluir los scripts SQL (*.sql*), ficheros de código Java (*.java*), ficheros de configuración (*.ini*, *.props*, *.conf*, . . .), . . .

También se incluyen en la categoría de ficheros de texto los que además incluyen información adicional (siempre en forma de texto) que permiten interpretar los datos del fichero de una manera u otra, añadiendo más información al mismo. Estos formatos son HTML, XML, JSON, . . .

En cualquier caso, desde Java siempre se podrán leer/escribir de la misma manera, según veremos a continuación. También veremos como pueden leerse/escribirse utilizando librerías aquellos ficheros de texto plano que contienen formato, como hemos comentado anteriormente, haciendo de esa forma mucho más fácil el trabajo.

Leer ficheros de texto plano

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public class LeerFichero {
    public static void main(String[] args) {
        String ruta = "archivo.txt";

        // try-with-resources cierra el fichero automáticamente
        try (BufferedReader br = new BufferedReader(new FileReader(ruta))) {
            String linea;

            // Lee línea a línea hasta llegar al final del archivo (null)
            while ((linea = br.readLine()) != null) {
                System.out.println(linea);
            }
        } catch (IOException e) {
            System.out.println("Error al leer el archivo: " + e.getMessage());
        }
    }
}
```

Escribir en ficheros de texto plano

```
import java.io.FileWriter;
import java.io.IOException;
```

```

public class EscribirFichero {
    public static void main(String[] args) {
        String texto = "Hola, este es un texto de prueba.";

        try {
            // El parámetro 'false' (o no ponerlo) sobrescribe el archivo si ya existe.
            // Si pones 'true' en su lugar, se añade el texto al final sin borrar lo anterior.
            FileWriter escritor = new FileWriter("archivo.txt");

            escritor.write(texto);
            escritor.close();

            System.out.println("Texto escrito con éxito.");
        } catch (IOException e) {
            System.out.println("Ocurrió un error al escribir el archivo.");
            e.printStackTrace();
        }
    }
}

```

Ficheros de propiedades (configuración)

En la API de Java se incluyen librerías para trabajar con los ficheros de configuración. Puesto que todos siguen un mismo patrón, es la librería la que se encarga de acceder al fichero a bajo nivel y el programador sólo tiene que indicar a que propiedad quiere acceder o que propiedad quiere modificar, sin tener que añadir nada de código para leer o escribir el fichero tal y como hemos visto en el punto anterior.

Un ejemplo de fichero de configuración de esta librería de java sería el fichero que sigue:

```

# Configuración del Servidor
server.port=8080
server.host=localhost
server.timeout=3000
# Conexión a la Base de Datos
db.url=jdbc:mysql://localhost:3306/mi_base_
datos db.user=admin
db.password=Secret123!

```

Las líneas que empieza por # son comentarios y son ignoradas por la clase

Escribir ficheros de configuración

Así, si queremos generar, desde Java, un fichero de configuración como el anterior:

```

import java.io.FileInputStream;

```

```

import java.io.FileOutputStream;
import java.io.IOException;
import java.util.Properties;

public class EjemploProperties {

    public static void main(String[] args) {
        String nombreArchivo = "configuracion.properties";

        // 1. ESCRIBIR PROPIEDADES
        try (FileOutputStream salida = new FileOutputStream(nombreArchivo)) {
            Properties propsEscritura = new Properties();

            // Asignar valores
            propsEscritura.setProperty("usuario", "admin");
            propsEscritura.setProperty("puerto", "8080");

            // Guardar en el fichero con un comentario opcional
            propsEscritura.store(salida, "Configuración del sistema");
            System.out.println("Archivo escrito correctamente.");

        } catch (IOException e) {
            System.out.println("Error al escribir el archivo: " + e.getMessage());
        }

        // 2. LEER PROPIEDADES
        try (FileInputStream entrada = new FileInputStream(nombreArchivo)) {
            Properties propsLectura = new Properties();

            // Cargar el archivo
            propsLectura.load(entrada);

            // Obtener valores por su clave
            String usuario = propsLectura.getProperty("usuario");
            String puerto = propsLectura.getProperty("puerto");

            System.out.println("Usuario leído: " + usuario);
            System.out.println("Puerto leído: " + puerto);

        } catch (IOException e) {
            System.out.println("Error al leer el archivo: " + e.getMessage());
        }
    }
}

```

Ficheros XML

Los ficheros XML permiten el intercambio de información entre aplicaciones utilizando para ello un fichero de texto plano al que se le pueden añadir etiquetas para darle significado a cada uno de los valores que se almacenan.

Por ejemplo, el siguiente fichero XML podría ser el resultado de volcar una Base de Datos sobre productos de una compañía, de forma que dicha información podría ahora leerse desde otra aplicación e incorporarla. Se puede ver como a parte de los datos de dichos productos (sus nombres, precios, . . .) aparece otra información en forma de etiquetas (entre los caracteres < y >) que permite dar significado a cada dato almacenado en el fichero. Así, podemos saber de qué estamos hablando y a qué corresponde cada valor.

Obviamente, tal y como ocurría con los ficheros de configuración, toda la información se tiene que pasar a texto para poder crear el fichero y reconvertida a su tipo original cuando se cargue de nuevo. Las mismas consideraciones que hemos tenido en cuenta antes nos servirán ahora (ver [aquí](#))

[productos.xml](#)

```
<?xml version="1.0" encoding="UTF-8" standalone="no">
<xml>
<productos>
  <producto>
    <nombre>Cereales</nombre>
    <precio>3.45</precio>
  </producto>
  <producto>
    <nombre>Colacao</nombre>
    <precio>1.45</precio>
  </producto>
  <producto>
    <nombre>Agua mineral</nombre>
    <precio>1.00</precio>
  </producto>
</productos>
</xml>
```

La clase Java que definiría cada uno de los objetos que se representan por el fichero XML anterior, sería la siguiente:

[Producto.java](#)

```
public class Producto {
  private String nombre;
  private float precio;

  public Producto(String nombre, float precio) {
    this.nombre = nombre;
    this.precio = precio;
  }

  public String getNombre () { return nombre; }
  public void setNombre (String nombre) { this.nombre = nombre; }

  public float getPrecio () { return precio; }
  public void setPrecio (float precio) { this.precio = precio; }
}
```

Escribir ficheros XML

Si ahora queremos generar el fichero XML con toda la información de una colección de objetos *Producto*, podemos utilizar el siguiente código:

```
...
documento = dom.createDocument(null, "xml", null);
Element raiz = document.createElement("productos");
documento.getDocumentElement().appendChild(raiz);
Element nodoProducto = null , nodoDatos = null ;
Text texto = null;

for (Producto producto : listaProductos) {
    nodoProducto = documento.createElement("producto");
    raiz.appendChild(nodoProducto);
    nodoDatos = documento.createElement("nombre");
    nodoProducto.appendChild(nodoDatos);
    texto = documento.createTextNode(producto.getNombre());
    nodoDatos.appendChild(texto);
    nodoDatos = documento.createElement("precio");
    nodoProducto.appendChild(nodoDatos);
    texto = documento.createTextNode(producto.getPrecio());
    nodoDatos.appendChild(texto);
}
...
```

Leer ficheros XML

Y si lo que queremos es leer un fichero XML y cargarlo como una colección Java de objetos *Producto*, el ejemplo siguiente muestra un breve ejemplo acerca de cómo acceder a la información de dicho fichero XML.

```
...
NodeList productos =
documento.getElementsByTagName("producto"); for (int i = 0; i <
productos.getLength(); i++) {
    Node producto = productos . item ( i ) ;
    Element elemento = ( Element ) producto ;
    System.out.println(elemento.getElementsByTagName("nombre").item(0)
        .getChildren().item(0).getNodeValue());
    System.out.println(elemento.getElementsByTagName("precio").item(0)
        .getChildren().item(0).getNodeValue());
}
```

Serialización de objetos

Serializar es el proceso por el cual un objeto en memoria pasa a transformarse en una estructura que pueda ser almacenada en un fichero (persistencia). Al proceso contrario le llamaremos *deserializar*.

Hay que tener en cuenta que durante el proceso de serialización, cada objeto se serializa a un fichero, por lo que si queremos almacenar todos los objetos de una aplicación, la idea más conveniente es tener todos éstos en alguna estructura compleja (y por comodidad dinámica) de forma que sea esta estructura la que serialicemos o deserialicemos para guardar o cargar los objetos de una aplicación. Estructuras como `ArrayList`, `Set` ó `HashMap` son clases muy utilizadas para trabajar de esta manera.

Serializar un objeto

```
import java.io.Serializable;

// La clase debe implementar Serializable
public class Producto implements Serializable {
    // Es una buena práctica definir este ID para el control de versiones de la clase
    private static final long serialVersionUID = 1L;

    private String nombre;
    private double precio;

    public Producto(String nombre, double precio) {
        this.nombre = nombre;
        this.precio = precio;
    }

    @Override
    public String toString() {
        return "Producto{nombre=\"" + nombre + "\", precio=" + precio + "\"}";
    }
}

import java.io.FileOutputStream;
import java.io.ObjectOutputStream;
import java.io.IOException;

public class Main {
    public static void main(String[] args) {
        // 1. Crear el objeto que queremos guardar
        Producto producto = new Producto("Portátil", 799.99);
        String rutaArchivo = "producto.dat";

        // 2. Utilizar try-with-resources para asegurar el cierre de archivos
        try (FileOutputStream fileOut = new FileOutputStream(rutaArchivo);
            ObjectOutputStream out = new ObjectOutputStream(fileOut)) {

            // 3. Grabar el objeto en el fichero
            out.writeObject(producto);
        }
    }
}
```

```
        System.out.println("Objeto Producto guardado correctamente en: " + rutaArchivo);  
    } catch (IOException e) {  
        System.err.println("Error al guardar el objeto: " + e.getMessage());  
    }  
}  
}
```